

InkClear: Pretrained Neural Cleanup and Offline Handwriting OCR

Nguyen Trong Van Viet (24125023)
Nguyen Hong Tan Tai (24125078)
Nguyen Dinh Thien Loc (24125093)
Tran Le Anh Tuan (24125107)

5 August 2026

Contents

1	Introduction	2
2	Product decisions	2
3	Project configuration	2
4	User workflow	2
5	Neural Clean pipeline	3
5.1	Model selection	3
5.2	Input, tiling, and rendering	3
5.3	Conversion validation	3
6	Offline OCR pipeline	3
7	Privacy and data handling	4
8	Interface design	4
9	Testing	4
9.1	Local tests	4
9.2	Instrumented tests	4
9.3	Scope of evaluation data	5
10	Build and verification	5
11	Limitations	5
12	AI usage statement	5
13	References	6
14	Conclusion	6

1 Introduction

InkClear is an Android application for photographed handwritten documents. The release described in this report combines two pretrained neural pipelines: a document-binarization network cleans shadows and uneven paper, while PP-OCRv5 Mobile detects and recognizes text. Both pipelines are bundled in the application and execute locally. The user starts OCR explicitly, reviews editable text on a dedicated screen, and can copy it or save a UTF-8 text file.

No model training is required on the user’s computer. The original deterministic enhancer remains available automatically when neural cleanup cannot start.

2 Product decisions

The agreed scope is English-first OCR, offline-only inference, one page at a time, explicit recognition after a button tap, a dedicated editable result screen, and plain-text export. Searchable PDF and model training are outside this release. The minimum Android version was allowed to change to satisfy the official mobile OCR SDK.

3 Project configuration

Item	Value
Project name	Android_AI_Integration
Application label	InkClear
Application ID	mobile_app.android_ai_integration
Minimum SDK	API 26 (Android 8.0)
Target SDK	API 36
Compile SDK	API 37.1
Gradle wrapper	9.6.1
Android Gradle Plugin	9.3.0
Java/Kotlin target	17
LiteRT runtime	TensorFlow Lite 2.17.0
OCR runtime	ONNX Runtime Android 1.21.1
Computer vision	Official OpenCV Android 4.13.0

Table 1: Implemented project configuration.

The application is primarily Java with XML layouts. A small Kotlin wrapper provides coroutine-based integration with the official Kotlin PaddleOCR SDK. Dependency versions are centralized in `gradle/libs.versions.toml`.

4 User workflow

1. The user selects an image or opens the optional one-page document scanner.
2. InkClear decodes and bounds the image to a maximum dimension of 1800 pixels.
3. Neural Clean runs in the background and displays its result. If loading or inference fails, the classical enhancer is used.
4. The user compares Original and Neural Clean, selects Natural, Grayscale, or Black and White, and adjusts strength.

5. The user taps Recognize text. PP-OCrv5 detects and recognizes lines locally.
6. A dedicated result activity shows the source preview, runtime metadata, confidence, and editable text.
7. The user copies text, saves a UTF-8 `.txt` file, or requests recognition again.

JPEG and image-only PDF export remain available for the cleaned page.

5 Neural Clean pipeline

5.1 Model selection

A large SBB cleanup SavedModel was evaluated first. Its float16 conversion remained approximately 75 MB and depended on `tf.ExtractImagePatches`, requiring TensorFlow Flex. That candidate was rejected for mobile deployment.

The selected model is the Apache-2.0 pretrained network from [sliedes/binarize](#), revision `f40863ed4f2869add585f8a133af29c37844ffee`. It has 6,501,345 parameters and was converted from TensorFlow SavedModel to a dynamically weight-quantized 6,906,912-byte LiteRT file.

5.2 Input, tiling, and rendering

The model accepts RGB float input with shape $1 \times 224 \times 224 \times 3$, using pixel values from 0 to 255, and produces one paper/ink logit per pixel. InkClear bounds the inference canvas to 896 pixels, divides it into non-overlapping 224-pixel tiles, applies a stable sigmoid and strength-controlled contrast, and upscales the mask to the source resolution.

Natural mode restores limited source chroma; Grayscale emits enhanced luminance; Black and White thresholds the neural mask. This one-pass mobile adaptation is faster than the upstream desktop method, which uses overlaps and rotations, but may lose detail in very small writing.

5.3 Conversion validation

The original TensorFlow and converted LiteRT models were compared on the handwritten DIBCO 2009 sample `dibco09_hand_H01.png`. The mean absolute logit difference was 0.10663 and the maximum was 0.67612. These values validate conversion similarity, not end-user cleanup quality. A separate Android instrumented test runs the packaged model end to end.

6 Offline OCR pipeline

The project vendors the official PaddleOCR Android SDK at revision:

`2661c7c0ef5c613e8f93c6e93b2e052399f0f854`

It bundles:

- PP-OCrv5 Mobile detection ONNX model, 4,826,518 bytes;
- PP-OCrv5 Mobile recognition ONNX model, 16,534,782 bytes;
- the recognition character configuration;
- ONNX Runtime Android and official OpenCV Android.

`OfflineOcrEngine` lazily creates the native engine only after the button tap. Detection finds line quadrilaterals; boxes are sorted in reading order; OpenCV performs perspective crops; recognition decodes each crop; and `OcrTextFormatter` normalizes whitespace while preserving line and paragraph structure.

Recognition owns a cancellable job. A request generation counter prevents stale text from opening after the image or enhancement settings have changed. The OCR engine releases ONNX sessions during activity destruction.

7 Privacy and data handling

All models are application assets. The source manifest explicitly removes `android.permission.INTERNET` and `ACCESS_NETWORK_STATE`, including declarations introduced by transitive libraries. An instrumented test verifies the merged installed package has no Internet permission.

The app does not log image pixels or recognized text, does not retain OCR history, and does not request broad storage access. JPEG, PDF, and TXT output use Android's system document picker. The optional Play-services scanner can be unavailable; the standard image picker is the fallback.

8 Interface design

The visual system uses a navy/blue palette, light paper surfaces, Neural Clean and Offline status chips, and a new adaptive launcher icon showing a sheet, handwriting, cleaned lines, and a sparkle. The OCR result screen is scroll-safe under compact displays and large accessibility scaling. It provides editable/selectable text, metadata, character count, Copy, TXT export, retry, and empty-result guidance.

9 Testing

9.1 Local tests

Local tests cover horizontal whitespace collapse, paragraph preservation, leading/trailing blank removal, empty input, Unicode preservation, and numerical stability of the neural sigmoid.

9.2 Instrumented tests

Five instrumented tests pass on the `Resizable_Experimental` Android Virtual Device:

1. bundled neural and OCR model assets are present and Internet permission is absent;
2. Recognize text is disabled before an image is ready;
3. result text is displayed, editable, scroll-safe, and updates its character count;
4. the real bundled PP-OCrv5 detector and recognizer execute end to end offline;
5. the real bundled LiteRT Neural Clean model executes end to end offline.

The OCR runtime fixture is PaddleOCR's official `android_ocr_benchmark_reference.png`. Synthetic English and Unicode strings exercise the formatter and UI.

9.3 Scope of evaluation data

The current fixtures prove conversion fidelity, packaging, privacy, lifecycle behavior, UI editing, and native inference. They do not provide a statistically meaningful handwriting-accuracy estimate. A future evaluation should use the IAM Handwriting Database for English line/word samples plus consented phone photographs covering pen, pencil, ruled paper, shadows, glare, cursive, and multiple writers. Character error rate and word error rate should be reported with writer-disjoint splits.

10 Build and verification

A valid JDK 17 or 21 and Android SDK Platform 37 are required. The official PaddleOCR Android SDK raises the minimum runtime to API 26. Typical verification commands are:

Listing 1: Build and test commands

```
.\gradlew.bat test
.\gradlew.bat lint
.\gradlew.bat assembleDebug
.\gradlew.bat :app:assembleDebugAndroidTest
.\gradlew.bat :app:connectedDebugAndroidTest
```

The debug APK is written to `app/build/outputs/apk/debug/app-debug.apk`. The first developer build needs Internet access to download Maven dependencies, but the installed application does not.

11 Limitations

- Dense cursive, touching lines, blur, glare, perspective, abbreviations, equations, and unusual symbols can reduce OCR accuracy.
- English is the release target; incidental recognition of other scripts is not guaranteed.
- Neural cleanup uses an 896-pixel inference bound for responsiveness, which can weaken very small strokes.
- Confidence is an average model score and is not a guarantee of correctness.
- The app processes one page at a time and does not create searchable PDFs.
- Bundled models and native runtimes substantially increase APK size.
- Physical-device latency and memory benchmarking remains necessary before production release.

12 AI usage statement

AI tools supported model research, conversion experiments, Android integration, UI concepts, test design, documentation, and troubleshooting. Every delivered path was checked through source inspection and automated builds. The converted model was numerically compared with its original TensorFlow source, and both neural pipelines were executed in Android instrumented tests. Model provenance, licenses, hashes, and known limits are recorded in `docs/MODELS.md`.

13 References

- PaddlePaddle, PaddleOCR repository and Android SDK.
- PaddlePaddle, PP-OCRv5 Mobile detection ONNX model.
- PaddlePaddle, PP-OCRv5 Mobile recognition ONNX model.
- S. Lieder, Document binarization using deep learning.
- Google AI Edge, LiteRT documentation.
- Microsoft, ONNX Runtime documentation.
- OpenCV, OpenCV for Android.

14 Conclusion

InkClear now contains meaningful pretrained neural functionality rather than a simulated AI label. It cleans documents with LiteRT, recognizes handwriting with PP-OCRv5, remains usable through a classical fallback, exposes an editable text workflow, and enforces an offline permission boundary. The next engineering milestone is a writer-disjoint English accuracy and performance benchmark on physical devices.